with strong unit typing), and financial APIs based on general unit and measurement implementation.

Components of the JavaMoney project

This JSR focuses on defining the interfaces and classes to be used for currencies and monetary amounts. Generally, the following areas are focused upon.

- *Core:* This has the data classes and interface representing currencies and monetary amounts.
- *Conversion:* This deals with exchange rates between currencies, and provides the API/SPI to perform the conversion of monetary amounts from one currency to another.
- *Format:* This defines APIs/SPIs for print formatting, and the parsing of currencies and monetary amounts, providing support for complex usage scenarios.
- *Extensions:* These add extended functionality like currency services, currency mapping, regions and validity services (historic data API).

Some of the key facilities of monetary functions are explained below.

BigMoney

This is an immutable money class and represents currency and BigDecimal amounts with any number of decimal places. An example is shown below:

```
BigMoney amount = BigMoney.parse("EUR 1.20567");
amount = amount.multipliedBy(2);          // EUR 2.41134
amount = amount.plusMajor(3);             // EUR 5.41134
amount = amount.plusMinor(5);             // EUR 5.46134
boolean negative = amount.isNegative();  // false
amount = amount.rounded(4, RoundingMode.UP);
String str = amount.toString();                 // "EUR 5.4614"
```

CurrencyUnit

This is a replacement of the existing currency class and allows applications to control currency data. It also includes three-digit numeric ISO code as shown below:

```
CurrencyUnit cur = CurrencyUnit.of("GBP");
int dp = cur.getDecimalPlaces();          // 2
String code = cur.getCurrencyCode();      // "GBP"
int ncode = cur.getNumericCode();       //
String str = cur.toString();               // "GBP"
```

Currency formatting

Printing currency values and units along with parsed data is also provided with a flexible builder, like Joda-Time and JSR 310. For example:

```
MoneyFormatterBuilder b = new MoneyFormatterBuilder();
b.appendCurrencyCode().appendLiteral(": ").appendAmount(
  MoneyAmountStyle.ASCII_DECIMAL_POINT_GROUP3_COMMA);
MoneyFormatter f = b.toFormatter();

String str = f.print(money);   // eg "GBP: 1,234.56"
```

Generally, the scope of the JSR is to target all general application types, such as:

- eCommerce
- Banking and finance
- Investment
- Insurance and pension
- Mobile and embedded payment

To summarise, the scope of JavaMoney can be defined as follows:

- Provides abstractions for currency and money suitable for all target applications.
- Enhances available currencies and adds support for non-ISO currencies as well.
- Supports complex calculation rules, including calculation precision and display precision.
- Supports regional rounding rules.
- Offers basic support for foreign exchange, extendable for more complex usage scenarios.
- A plain text representation of money.
- Printing/parsing to the string.
- Support for pluralisation.
- Provides APIs for historic access of currencies and exchange rates.
- Defines a flexible query API to support complex usage scenarios for exchange rates and currencies.
- Stores a value in multiple currencies (a single value type that effectively maps the currency to the amount, where the amounts are all notionally the same by some form of currency conversion).
- Large exchange rate system (as in a system suitable for trading, such as bid/ask).

Nevertheless, there are some limitations on this front:

- It is not planned to target low latency such as algorithmic trading applications, though we should try to keep performance in mind to accommodate such applications.
- Non-decimal currencies.

JavaMoney is included with Java 9, with backward compatibility as well as a pluggable library. It is being developed under the standard JSR Spec License and Apache 2.0 License for RI (Reference Implementation) and TCK (Technology Compatibility Kit). These two modules will be developed and distributed as standalone modules and RI, and bundled with the Java 8 SE distribution.

By: Magesh Kasthuri

The author is a senior distinguished member of the technical staff at Wipro. He is an enterprise architect in BFSI. He has authored a series of articles on Docker based DevOps in *OSFY* earlier, and can be reached at *magesh.kasthuri@wipro.com*.